



PERTEMUAN 4

Control Statements: Logical Operator

Esensi Iterasi Counter (Penghitung Terkendali)

Untuk memformalkan elemen yang diperlukan dalam melakukan iterasi yang terkendali, terdapat empat komponen utama yang harus dipahami. Keempat elemen ini bekerja bersama untuk mengendalikan alur perulangan secara terstruktur dan dapat diprediksi.

1

Variabel Kontrol

Disebut juga penghitung loop. Variabel ini digunakan untuk melacak berapa kali loop telah berjalan.

2

Nilai Awal

Nilai inisialisasi dari variabel kontrol sebelum loop pertama kali dijalankan.

3

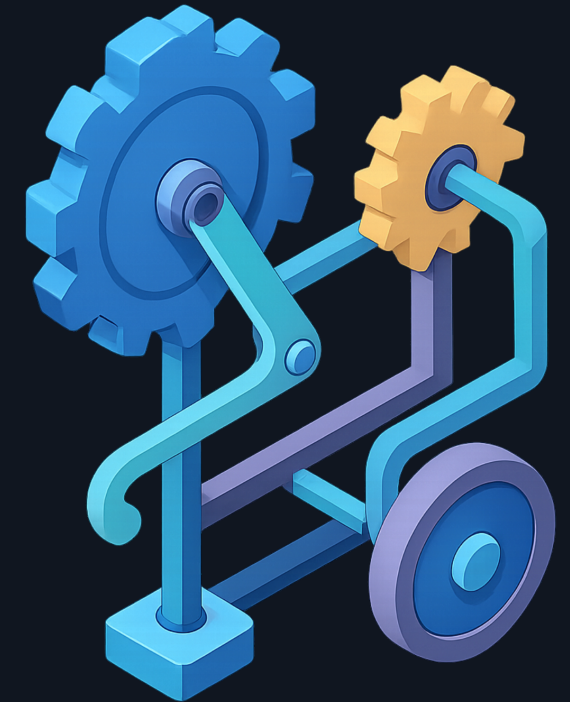
Kenaikan (Increment)

Nilai yang digunakan untuk memodifikasi variabel kontrol setiap kali satu iterasi loop selesai dilakukan.

4

Kondisi Kelanjutan

Ekspresi boolean yang menentukan apakah loop harus terus berjalan atau berhenti pada setiap iterasi.



Contoh: Iterasi Counter

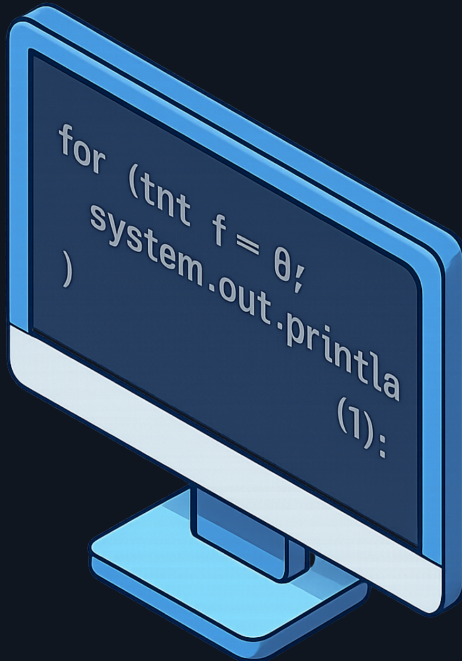
Berikut adalah contoh implementasi iterasi counter menggunakan pernyataan `while` di Java. Perhatikan bagaimana keempat elemen iterasi — variabel kontrol, nilai awal, kenaikan, dan kondisi kelanjutan — diterapkan secara eksplisit dalam kode.

```
int counter = 1;           // 1. Variabel kontrol + 2. Nilai awal

while (counter <= 10) {    // 4. Kondisi kelanjutan loop
    System.out.println("Counter: " + counter);
    counter++;             // 3. Kenaikan variabel kontrol
}
```

- ❶ Pada contoh ini, loop akan mencetak angka 1 hingga 10, lalu berhenti ketika nilai `counter` melebihi 10. Setiap iterasi menambah nilai `counter` sebesar 1.

For Iteration Statement



Pernyataan `while` dapat digunakan untuk mengimplementasikan loop yang dikendalikan oleh penghitung. Namun, Java juga menyediakan **for iteration statement** yang jauh lebih ringkas — menentukan semua detail iterasi yang dikendalikan penghitung dalam **satu baris kode** saja.

Struktur For Statement

```
for (inisialisasi; kondisi;  
    kenaikan) {  
    // body loop  
}
```

Keunggulan For Statement

- Semua elemen kontrol loop ditulis dalam satu baris
- Kode lebih ringkas dan mudah dibaca
- Mengurangi risiko lupa menulis kenaikan variabel
- Cocok untuk iterasi dengan jumlah yang sudah diketahui

UML Activity Diagram untuk For Statement

Diagram berikut menggambarkan alur eksekusi pernyataan `for` di Java. Perhatikan urutan evaluasi: inisialisasi hanya terjadi sekali, kemudian kondisi diperiksa, body dieksekusi, lalu kenaikan dilakukan sebelum kondisi dievaluasi kembali.



Jika kondisi bernilai `false` sejak awal, body loop tidak akan pernah dieksekusi. Inilah yang membedakan `for` dan `while` dari `do...while`.

Contoh Menggunakan For

Berikut contoh penggunaan `for` statement untuk mencetak angka 1 hingga 10. Ketiga elemen kontrol loop ditulis langsung dalam tanda kurung `for`, membuat kode lebih terstruktur dan mudah dipahami.

```
public class ForExample {  
    public static void main(String[] args) {  
  
        // for(inisialisasi; kondisi; kenaikan)  
        for (int i = 1; i <= 10; i++) {  
            System.out.printf("Iterasi ke-%d\n", i);  
        }  
  
        System.out.println("Loop selesai!");  
    }  
}
```

Penjelasan Elemen

- `int i = 1` → Inisialisasi variabel kontrol
- `i <= 10` → Kondisi kelanjutan loop
- `i++` → Kenaikan setiap iterasi

Output Program

```
Iterasi ke-1  
Iterasi ke-2  
...  
Iterasi ke-10  
Loop selesai!
```

Aplikasi: Menjumlahkan Bilangan Bulat Genap dari 2 hingga 20

Menggunakan pernyataan `for` untuk menjumlahkan bilangan bulat genap dari 2 hingga 20 dan menyimpan hasilnya dalam variabel `int` bernama `total`. Ini adalah contoh klasik pemanfaatan `for` dengan kenaikan 2.

```
public class SumEvenNumbers {  
    public static void main(String[] args) {  
        int total = 0;  
  
        for (int number = 2; number <= 20; number += 2) {  
            total += number;  
        }  
  
        System.out.printf("Jumlah bilangan genap 2-20 = %d\n", total);  
    }  
}
```

✓ Output: **Jumlah bilangan genap 2-20 = 110**. Kenaikan `number += 2` memastikan hanya bilangan genap yang dijumlahkan.



Kasus Menghitung Bunga Majemuk (Investasi)

Selanjutnya, mari kita gunakan pernyataan for untuk menghitung bunga majemuk. Pertimbangkan masalah berikut:

Seseorang menginvestasikan Rp 1.000 dalam rekening tabungan dengan bunga 5% per tahun. Berapakah saldo rekening pada akhir setiap tahun selama 10 tahun, dengan asumsi semua bunga dibiarkan dalam rekening?

Formula bunga majemuk yang digunakan adalah:

$$a = p \times (1 + r)^n$$

a

Jumlah akhir setelah n tahun

p

Jumlah pokok yang diinvestasikan

r

Tingkat bunga tahunan

n

Jumlah tahun

Kode Program: Bunga Majemuk

Berikut implementasi lengkap perhitungan bunga majemuk menggunakan `for statement` di Java. Program ini menampilkan saldo akhir setiap tahun selama 10 tahun.

```
public class Interest {  
    public static void main(String[] args) {  
        double principal = 1000.0; // investasi awal  
        double rate = 0.05;        // suku bunga 5%  
  
        System.out.printf("%-4s%21s%n", "Tahun", "Jumlah di Rekening");  
  
        for (int year = 1; year <= 10; year++) {  
            double amount = principal * Math.pow(1.0 + rate, year);  
            System.out.printf("%-4d%21.2f%n", year, amount);  
        }  
    }  
}
```

 `Math.pow(basis, eksponen)` adalah metode bawaan Java untuk menghitung nilai pangkat. Di sini digunakan untuk menghitung $(1 + r)^n$.

Analisis Kode: Jelaskan Kode Berikut

Perhatikan potongan kode berikut dan analisis apa yang dilakukannya. Identifikasi variabel, kondisi, dan keluaran yang dihasilkan.

```
public class Analisis {  
    public static void main(String[] args) {  
        int x = 1;  
  
        while (x <= 5) {  
            System.out.printf("%d ", x * x);  
            x++;  
        }  
  
        System.out.println();  
    }  
}
```

Pertanyaan Analisis

- Apa yang disimpan oleh variabel x?
- Kapan loop berhenti berjalan?
- Apa output yang dihasilkan program ini?
- Apa fungsi dari ekspresi `x * x`?

Jawaban

Program mencetak kuadrat dari angka 1 hingga 5.

Output: 1 4 9 16 25

Loop berhenti ketika `x` bernilai lebih dari 5.

Do...While Iteration Statement

Pernyataan `do...while` adalah variasi dari `while` yang memiliki satu perbedaan mendasar: kondisi kelanjutan loop dievaluasi **setelah** body loop dieksekusi, bukan sebelumnya.

While Statement

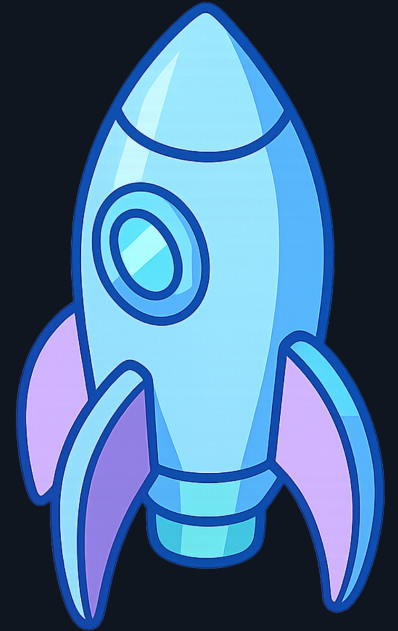
Menguji kondisi di **awal** loop. Jika kondisi salah sejak awal, body tidak pernah dieksekusi sama sekali.

Do...While Statement

Menguji kondisi di **akhir** loop. Body selalu dieksekusi **minimal satu kali**, berapapun nilai kondisinya.

```
do {  
    // body loop – selalu dieksekusi minimal sekali  
} while (kondisi); // kondisi diperiksa di sini
```

Ketika pernyataan `do...while` berakhir, eksekusi berlanjut dengan pernyataan berikutnya secara berurutan.



UML Activity Diagram: Do...While

Perbedaan utama diagram alur do...while dibandingkan while biasa adalah posisi pengecekan kondisi yang berada **setelah** eksekusi body, bukan sebelumnya. Hal ini menjamin body selalu berjalan minimal sekali.



```
int counter = 1;

do {
    System.out.println("Nilai counter: " + counter);
    counter++;
} while (counter <= 5);
```

Gunakan **do...while** ketika body loop harus dijalankan setidaknya satu kali, misalnya saat meminta input dari pengguna hingga input valid.

Switch Multiple-Selection Statement

Pernyataan `switch` memungkinkan program melakukan tindakan berbeda berdasarkan nilai dari sebuah ekspresi. Ini adalah alternatif yang lebih bersih dibandingkan rangkaian `if-else if` yang panjang.

Tipe Data yang Didukung

- `byte`, `short`, `int`
- `char` (karakter tunggal)
- `String` (sejak Java 7)
- *Catatan: tipe*



Contoh Kode Program: Switch

Berikut contoh penggunaan `switch` untuk menentukan grade berdasarkan nilai numerik yang dimasukkan pengguna. Setiap `case` menangani rentang nilai tertentu menggunakan fall-through atau pengelompokan case.

```
import java.util.Scanner;

public class SwitchGrade {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan nilai (0-100): ");
        int nilai = input.nextInt();

        switch (nilai / 10) {
            case 10: case 9:
                System.out.println("Grade: A"); break;
            case 8:
                System.out.println("Grade: B"); break;
            case 7:
                System.out.println("Grade: C"); break;
            case 6:
                System.out.println("Grade: D"); break;
            default:
                System.out.println("Grade: F");
        }
    }
}
```

Analisis: Switch dengan String

Sejak Java 7, `switch` mendukung tipe data `String`. Berikut contoh penggunaan `switch` dengan ekspresi berupa nama hari dalam seminggu.

```
public class SwitchHari {  
    public static void main(String[] args) {  
        String hari = "Senin";  
  
        switch (hari) {  
            case "Senin":  
            case "Selasa":  
            case "Rabu":  
            case "Kamis":  
            case "Jumat":  
                System.out.println("Hari kerja"); break;  
            case "Sabtu":  
            case "Minggu":  
                System.out.println("Akhir pekan"); break;  
            default:  
                System.out.println("Hari tidak dikenal");  
        }  
    }  
}
```

 Beberapa `case` dapat dikelompokkan tanpa `break` di antara mereka untuk berbagi aksi yang sama — ini disebut **fall-through**.

Switch Statement: UML Activity Diagram

Diagram UML berikut menggambarkan alur keputusan dari pernyataan `switch`. Setiap `case` mewakili satu jalur eksekusi yang berbeda, dan blok `default` menangani semua nilai yang tidak cocok dengan `case` manapun.



Tanpa pernyataan `break` di akhir setiap `case`, eksekusi akan terus mengalir ke `case` berikutnya — perilaku ini dikenal sebagai **fall-through** dan bisa disengaja atau menjadi bug jika tidak hati-hati.

Break Statement

Pernyataan `break` digunakan untuk keluar secara langsung dan segera dari sebuah struktur kontrol. Ketika dieksekusi di dalam `while`, `for`, `do...while`, atau `switch`, program langsung berhenti dari pernyataan tersebut dan melanjutkan eksekusi ke pernyataan pertama setelah blok kontrol.

Digunakan Dalam

- Pernyataan `while`
- Pernyataan `for`
- Pernyataan `do...while`
- Pernyataan `switch`

Contoh Penggunaan

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        break; // keluar saat i = 5  
    }  
    System.out.println(i);  
}  
// Output: 1 2 3 4
```

⚠️ Gunakan `break` dengan bijak. Penggunaan berlebihan dapat membuat logika program sulit diikuti dan dipahami.



Continue Statement

Berbeda dengan `break` yang menghentikan loop sepenuhnya, pernyataan `continue` hanya melewati sisa pernyataan di dalam body loop untuk iterasi saat ini, lalu langsung melanjutkan ke iterasi berikutnya.



1 Fungsi Utama

Melompati pernyataan yang tersisa di badan loop dan melanjutkan dengan iterasi loop berikutnya tanpa menghentikan loop.

2 Pada While & Do...While

Program mengevaluasi tes kelanjutan loop **segera setelah** pernyataan `continue` dijalankan.

3 Pada For Statement

Ekspresi kenaikan dijalankan terlebih dahulu, kemudian program mengevaluasi uji kelanjutan loop.

4 Perbedaan dengan Break

`break` menghentikan loop sepenuhnya; `continue` hanya melewati satu iterasi dan loop tetap berjalan.

Contoh Kode Program: Continue

Berikut contoh penggunaan `continue` untuk melewati bilangan ganjil dan hanya mencetak bilangan genap dari 1 hingga 10. Perhatikan bahwa loop tetap berjalan penuh meskipun beberapa iterasi dilewati.

```
public class ContinueExample {  
    public static void main(String[] args) {  
  
        for (int i = 1; i <= 10; i++) {  
            if (i % 2 != 0) {    // jika i adalah bilangan ganjil  
                continue;      // lewati iterasi ini  
            }  
            System.out.printf("%d ", i);  
        }  
  
        System.out.println("\nLoop selesai.");  
    }  
}
```

Output Program

```
2 4 6 8 10  
Loop selesai.
```

Cara Kerja

Ketika `i` bernilai ganjil, `continue` membuat program melompat langsung ke ekspresi kenaikan `i++`, melewati perintah `printf`.

Logical Operators

Java menyediakan beberapa operator logika untuk menggabungkan atau membalikkan kondisi boolean. Memahami operator ini sangat penting untuk membangun kondisi yang kompleks dalam program.

Conditional AND (&&)

Bernilai `true` hanya jika **kedua kondisi** bernilai `true`. Menggunakan evaluasi hubung-singkat: jika kondisi pertama `false`, kondisi kedua tidak dievaluasi.

Conditional OR (||)

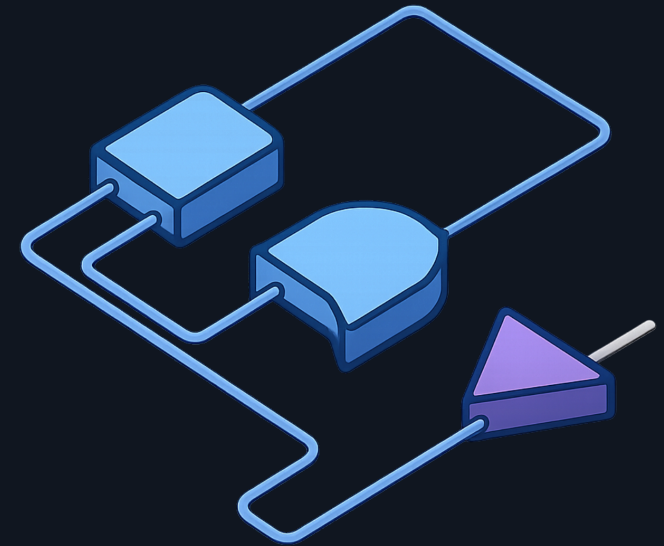
Bernilai `true` jika **salah satu atau kedua kondisi** bernilai `true`. Jika kondisi pertama `true`, kondisi kedua tidak dievaluasi (hubung-singkat).

Boolean Logical AND (&) dan OR (|)

Identik dengan `&&` dan `||`, tetapi **selalu mengevaluasi kedua operan** — tidak melakukan evaluasi hubung-singkat.

Logical Negation (!)

Operator NOT membalikkan nilai boolean suatu kondisi. Jika kondisi bernilai `true`, maka `!kondisi` bernilai `false`, dan sebaliknya.



Contoh Logical Operator

Berikut contoh implementasi berbagai logical operator dalam Java. Perhatikan bagaimana setiap operator menghasilkan nilai boolean yang berbeda berdasarkan kombinasi kondisi.

```
public class LogicalOperators {
    public static void main(String[] args) {
        int a = 10, b = 5, c = 10;

        // AND (&&): kedua kondisi harus true
        System.out.println(a > b && a == c); // true && true = true
        System.out.println(a > b && b == c); // true && false = false

        // OR (||): salah satu kondisi cukup true
        System.out.println(a > b || b == c); // true || false = true
        System.out.println(b > a || b == c); // false || false = false

        // NOT (!): membalik nilai boolean
        System.out.println(!(a == b)); // !(false) = true
        System.out.println(!(a == c)); // !(true) = false

        // Boolean AND (&): evaluasi kedua operan
        System.out.println(a > b & b == c); // true & false = false
    }
}
```

Analisis Kode!

Coba analisis dan prediksi output dari kode program berikut sebelum menjalankannya. Identifikasi operator logika yang digunakan dan telusuri alur eksekusinya baris per baris.

```
public class AnalisisLogika {  
    public static void main(String[] args) {  
        int x = 7, y = 4, z = 7;  
  
        if (x > y && x == z) {  
            System.out.println("Kondisi A terpenuhi");  
        }  
  
        if (x > y || y > z) {  
            System.out.println("Kondisi B terpenuhi");  
        }  
  
        if (!(y == z)) {  
            System.out.println("Kondisi C terpenuhi");  
        }  
  
        for (int i = 1; i <= 5; i++) {  
            if (i == 3) continue;  
            if (i == 5) break;  
            System.out.println("i = " + i);  
        }  
    }  
}
```

❓ Apa output dari program di atas? Kondisi mana yang terpenuhi? Angka berapa saja yang dicetak oleh loop for? Diskusikan bersama!